

CONCLUSION · DOWNLOAD · TEST GUIDE

AlumniConnect

Alumni Engagement & Career Referral Platform

Status: All 6 phases complete · **Project:** alumni-platform

Tech: Next.js 14 · React Native/Expo · Express · Prisma · PostgreSQL + pgvector · JWT · SendGrid · Twilio ·
Cloudinary · OpenAI

Deliverable: Full-stack web + mobile platform, ready to deploy

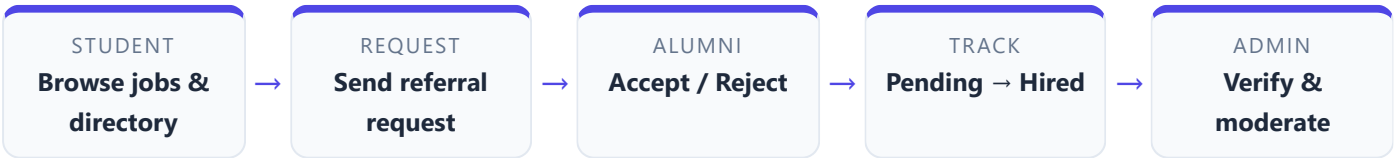
1 Conclusion — Where the Project Stands

AlumniConnect is **complete** — **all 6 phases done**. The core referral engine, engagement features, admin tooling, comms/file pipeline, AI matching, the native mobile app, and the deployment playbook are all implemented and verified.

Feature Checklist

Module	Status	What's included
Auth & Roles	Done	Register/login (JWT+bcrypt), 4 roles, profile + password change
Alumni Directory	Done	Search/filter by batch, dept, company, location; profile pages
Job Board & Referrals	Done	Post jobs, request referrals (resume + note), status tracking, notifications
Success Stories	Done	Submit, admin approve/feature, Spotlight Wall
Events + RSVP	Done	Create events, capacity limits, RSVP/cancel, attendee list
Announcements	Done	Faculty/admin post notices
Notifications	Done	In-app bell, unread badge, mark read/all
Admin Suite	Done	Analytics dashboard, alumni verification, CSV import, story review
Email / WhatsApp	Done	SendGrid welcome/referral emails + Twilio alerts (console mode without keys)
Resume Upload	Done	Cloudinary or local storage, 5 MB, in referral modal + profile
AI Smart Matching	Done	"Top 5 Alumni for You" — pgvector + OpenAI (local fallback), HNSW index
Mobile App	Done	Expo/React Native — login, jobs, referrals, stories, events, AI matches, profile
Deployment	Done	Supabase + Railway/Render + Vercel + EAS Build → Play Store / TestFlight (see docs/DEPLOYMENT.md)

Platform Flow



Architecture: Web (Next.js) and mobile (Expo/React Native) both talk to a single Express REST API over HTTPS with JWT. Prisma ORM maps to PostgreSQL with the **pgvector** extension for AI matching (OpenAI embeddings, local fallback). External services (Cloudinary, SendGrid, Twilio) plug in behind the API.

2 How to Download the Project

The entire project lives in the folder **C:\Users\vishw\Documents\alumni-platform**. Three ways to get a copy:

Option A — Download the folder

Copy the **alumni-platform** folder (or zip it: right-click → Send to → Compressed folder). The source is under **apps/**, **prisma/**, **packages/**, **docs/**.

Option B — Initialize a git repo and push to GitHub

```
cd C:\Users\vishw\Documents\alumni-platform
git init
git add -A
git commit -m "AlumniConnect - all 6 phases (web + api + mobile)"
# create a repo on github.com, then:
git remote add origin https://github.com/<you>/alumni-connect.git
git push -u origin main
```

Option C — ZIP archive (for sharing)

```
Compress-Archive -Path "C:\Users\vishw\Documents\alumni-platform\*" -DestinationPath
"C:\Users\vishw\Documents\alumni-platform.zip" -Force
```

Share the .zip — the recipient follows Section 3 to run it.

What's inside

```
alumni-platform/
├─ apps/
│  ├─ api/           # Express + Prisma backend (port 4000)
│  ├─ web/           # Next.js 14 frontend (port 3000)
│  └─ mobile/        # React Native / Expo app (Expo Go)
├─ prisma/
│  ├─ schema.prisma
│  └─ migrations/    # initial migration (all tables) + pgvector
├─ packages/shared-types/
├─ docs/             # PDFs + phase change logs + DEPLOYMENT.md
└─ README.md
```

3 Local Setup (one-time)

1. Install PostgreSQL

Install locally (postgresql.org) or create a free server at Neon/Supabase. Note the connection string.

2. Configure environment

```
copy apps\api\.env.example apps\api\.env
# edit apps/api/.env → set DATABASE_URL to your Postgres:
DATABASE_URL="postgresql://user:pass@localhost:5432/alumni_connect?schema=public"
```

The web app has no required env vars (defaults to `http://localhost:4000`).

3. Install dependencies

```
cd apps\api
npm install
cd ..\web
npm install
cd ..\mobile
npm install      # optional – only if testing the mobile app
```

4. Create the database + apply migrations

```
cd apps\api
npm run prisma:deploy    # applies prisma/migrations (creates all tables)
```

5. Seed demo accounts

```
npm run seed            # creates admin + demo alumni + demo student + AI embeddings
```

4 How to Test It

Step 1 — Start the servers

```
# Terminal 1
cd apps\api && npm run dev      → http://localhost:4000
# Terminal 2
cd apps\web && npm run dev      → http://localhost:3000
```

Open <http://localhost:3000> in a browser.

Step 2 — Demo logins

Role	Email	Password
Admin	admin@college.edu	Admin@12345
Alumni	alumni@college.edu	Alumni@12345
Student	student@college.edu	Student@12345

Phase 4 note: Emails & WhatsApp print to the API terminal as placeholders when no API keys are set (still exercises the full flow). Resumes upload to `apps/api/uploads/` locally.

Step 3 — Walk-through (try in this order)

1. **Alumni:** Post a Job → Jobs / Post a Job
2. **Student:** Browse Jobs → open the job → Request Referral → **upload a resume file** (or paste a link) + note
3. **Alumni:** My Referrals → Received tab → Accept → Mark as Referred — **watch the API terminal log the email & WhatsApp placeholders** + the bell light up
4. **Student:** My Referrals → Sent tab → status now shows the new stage
5. **Alumni:** Stories → Share Your Story → submit
6. **Admin:** Admin → Story Review → Approve / Approve & Feature → **alumni receives an approval email placeholder**
7. **Alumni/Student:** Stories → the story is now live on the Spotlight Wall
8. **Admin:** Events → Create Event → then as a student RSVP from the Events list
9. **Faculty/Admin:** Announcements → Post Announcement
10. **Admin:** Admin → Analytics Dashboard → verify alumni → CSV Import (download template, upload, see summary) — **welcome emails logged in API terminal**
11. **Student:** Profile → add skills + interests + phone → save → **AI Matches** → **Re-match me** → see Top 5 Alumni with match % → View Profile
12. **Admin:** Admin → Analytics → **Sync AI embeddings** → counts shown
13. **All:** Edit profile under the avatar → check the notification bell dropdown

Step 4 — Quick API check

```
GET http://localhost:4000/health → {"status":"ok", ... }
```

Step 5 — Mobile app (Expo Go)

```
cd apps\mobile  
npm install  
npx expo start → scan the QR with Expo Go
```

Android emulator uses 10.0.2.2:4000, iOS simulator uses localhost:4000. For a physical phone, set API_URL to your PC's LAN IP (e.g. http://192.168.1.50:4000) before starting. Log in with the same demo accounts and try: Job Board → Request Referral · Referrals tab (Accept/Referred) · Events RSVP · AI Matches (students) · Profile → save skills.

Step 6 — Deploy (production)

Follow **docs/DEPLOYMENT.md**: Supabase (managed Postgres only) → npm run prisma:deploy + npm run seed → API on Railway/Render → web on Vercel → eas build for the mobile APK/IPA → Google Play / TestFlight. Remember Cloudinary keys are required in production (local uploads are wiped on redeploy).

Troubleshooting: If pages show "Loading" forever or the bell can't load, the API isn't running (Step 1). If login fails, re-run npm run seed. If the mobile app can't reach the API, check the phone can reach the API URL (never use localhost on a physical device). To reset everything, drop and recreate the database, then re-run migrate + seed.